TEST PLAN — E-COMMERCE STORE

COMP.SE.200-2026-2027-1 Software Testing | Part 1 — Test Plan Group 31 | Pham Thanh(154888686) | Georgios Malezoglou (154731995) Version: 1.0 | **Date:** 7 November 2026

Definitions and abbreviations

Term	Meaning
SUT	System under test: the selected utility-library source files
E2E	End-to-end; a complete user workflow across system components
CI	Continuous integration; automated verification on code changes
REST/API	Representational State Transfer / application programming interface
Coverage	Proportion of instrumented statements or branches exercised by tests
Stub	Controlled substitute for a dependency during a test
Unit test	A test of one small unit of code in isolation
Quality gate	A condition that must be satisfied before testing or a change is accepted

1. Introduction

1.1 Purpose and deliverables

This plan explains what will be tested and why, how the tests will be designed, and how their results will be recorded and presented. It also explains how the results will help on deciding whether a helper functions library with no documentation can be reused in a food product e-commerce shop. It includes three E2E scenarios, 10 selected library functions in a priority order, a reproducible test approach, reporting and defect procedures, quality gates, detailed unit-test designs, and additional system-level testing recommendations. This is what Part 1 or this project will deliver, while part 2 includes the implementation of unit tests for the selected functions, GitHub Actions and Coveralls integration [1].

1.2 Application context and risk

Customers search and filter products based on name, category, price, ingredients and the product producer. Customers can navigate through the results (splitted in different pages), update their cart and use an external provider for payment. Producers can add products manually or through an existing bulk-import portal. Inputs like name, price and producer are mandatory while category and contents are not. The frontend has to validate prices, descriptions and order inputs. A REST backend takes care of third-party services and catalogue operations [1]. Products may hide by wrong filters, totals may be affected due to numeric conversion errors and poor validation may admit invalid product records. The library is available in a project that has be abandoned and documentation is missing, so a developer should not guess what each function based on the name alone.

2. Context of testing

2.1 Three representative E2E scenarios

S1 — Product search, filtering and pagination

Prerequisites: Catalog has several producers, optional attributes that could be missing, and way too many matches to be displayed in one page. **Basic workflow:** (1) user accesses catalog; (2) user inputs the query into the search box; (3) frontend validates input and sends a REST request; (4) backend responds with matches; (5) user applies filters according to category, price, content, or producer; (6) system combines filters; (7) user navigates between pages; (8) matches are displayed on a page based on filters applied. **Variations:** empty/blank search, no matches, optional attributes missing, last/incorrect page, or incorrect price interval. **Expected output:** only valid matches are shown, paging works, blank output is justified. **Possible utilities:** `filter`, `every`, `toNumber`, `isEmpty`, `defaultTo`, `chunk`, `slice`. **Risk:** incorrect filters or pages numbers lead to hiding or misrepresentation of inventory. Activity diagram: refer to Appendix A.

S2 — Cart, quantities and checkout

Preconditions: The prices of products are correct; access to the cart and the checkout. **Flow:** (1) products selected by the customer; (2) add/remove a product to/from the cart; (3) change the quantity; (4) calculation

of subtotals and total by the cart after each action; (5) cart overview; (6) sending checkout data from the backend to an external provider; (7) showing the outcome on UI. **Cases to consider:** incorrect, null, negative or non-integer quantity; empty cart; unavailable provider; duplicate submission. **Expected results:** accurate cart state and proper formatting of money values without false indication of payment success. **Candidate utilities:** `add`, `reduce`, `toNumber` (and `toInteger`, `clamp` as alternatives). **Risk:** incorrect totals or duplicate order. Activity Diagram: see Appendix B. The general `add` utility is not reliable for processing decimal money; special tests are needed for money utilities.

S3 — Producer creation and bulk import

Prerequisites: Access by the producer of the entry form or import portal; mandatory field requirements. **Main flow:** (1) choice of single entry or bulk import by the producer; (2) providing product name, price, producer, and optional additional information; (3) validation of input by the frontend/importer; (4) invalid entries reported per the import policy; (5) valid entries stored using REST; (6) producer verifies entries in the catalogue. **Variants:** missing mandatory field, optional fields null values, incorrectly formatted prices, mixed invalid/valid entries, duplicates, and bulk imports. **Expected results:** No silent addition of invalid products, successful import and correct catalogue. **Candidate utilities:** `toNumber`, `isEmpty`, `every`, `defaultTo`, `countBy`; `upperFirst` is an unused candidate for formatting purposes. **Risk:** Corrupted data in catalogue affects search and purchase. Activity diagram: Appendix C.

2.2 Scope and risk-based selection

Only ten `src` files at the highest level will be subjected to unit test. The hidden `.internal` folder is assumed to be error-free and hence will neither be tested nor covered, according to the instructions [1]. The user interface for the application, REST back-end service, and the interaction with the payment provider are beyond the scope of the provided implementation and their suggested tests are listed in §4.3.

The team chose the ten files above based on three criteria between 1-3 in total: **scenarios coverage** (number of S1-S3 scenarios that can be used), **impact of failure** (consequences of a possible error) and **estimated defect likelihood** (complexity and corner cases). **Total = coverage + impact + likelihood.** It is not about actual defect probability but rather an estimation for planning purposes. In case of ties, the priority goes first by the business impact, then by the coverage, and finally by the relevance to implementation chosen by the group. Scores and the unselected candidates are recorded in Appendix D.

Selected file	Scenarios	Primary test focus
`toNumber.js`	S1, S2, S3	Numeric coercion, invalid values and prices
`filter.js`	S1	Predicate correctness, empty and mixed arrays
`chunk.js`	S1	Page sizes, remainder and boundary inputs
`isEmpty.js`	S1, S2, S3	Empty strings, arrays, objects and null-like data
`add.js`	S2	Arithmetic, coercion and precision limitations
`countBy.js`	S3	Group counts and duplicate detection support
`reduce.js`	S2	Accumulation, initial value and empty collections
`every.js`	S1, S3	All-condition predicates and empty collections
`defaultTo.js`	S1, S3	Missing optional values and valid falsy values
`slice.js`	S1	Start/end bounds and result subsets

The files `clamp.js`, `toInteger.js`, `toString.js`, and `upperFirst.js` were considered but not included due to the file limit of ten. `map.js` and `toFinite.js` are other possible choices but not included in the final selection of the team. This is a **selection rationale** that the team has to verify the actual exports of the functions prior to Part 2. The library helper does not perform the search, check out, or producer validation.

2.3 Assumptions and limitations

`.internal` functions within the library are deemed safe solely based on the fact that the assignment allows for such an assumption to be made, while no other `src` functions may be considered so. Expected results need to be based on the function semantics known for certain rather than on theoretical applications. If any of the exported helpers forcibly manipulates the input or outputs IEEE-754 floating point numbers, then such actions need to be mentioned along with the potential application-based issues. There will be no application code, backend interface, sandboxed payments, nor example products provided [1].

3. Tools, reporting and quality controls

3.1 Environment and toolchain

The proposed **Part 2** stack is Node.js/npm, Git/GitHub, **Jest** for test execution/assertions, **Jest's coverage reporting** for local statement/branch data and LCOV output, **GitHub Actions** for CI and **Coveralls** for published coverage. This is just a planned setup. We have not yet downloaded or used these tools. Before setting up Jest, we will look at `package.json` to see how the library's modules are written (ESM or CommonJS). If something does not match, it will be documented in the final report. We will not secretly change the library's code. `c8` is another coverage tool, but we do not need it, since Jest already does this. Chai, Mocha and testdouble are not needed for what we want to achieve. GitHub Actions and Coveralls will be set up in Part 2, not now [1,3–6].

Tool	Role	Reason for choosing
Node.js / npm	JavaScript runtime and dependency management.	Required to execute the JavaScript tests and install Jest and other project dependencies.
Git / GitHub	Version control, test code storage, collaboration, and defect tracking through GitHub Issues.	Already required for the assignment submission and provides integration with the CI pipeline.
Jest	Unit test execution and assertions.	The team has previous experience with Jest. It provides a test runner, assertions, and built-in coverage support.
Jest Coverage	Measure local statement and branch coverage and generate LCOV reports.	Integrated into Jest, avoiding the need for a separate coverage library.
GitHub Actions	Automatically execute tests on pushes and pull requests (Part 2).	Integrated with GitHub and does not require a separate CI server.
Coveralls	Publish coverage results and display coverage information for individual source files (Part 2).	Accepts LCOV reports generated by Jest and provides a centralised view of test coverage.

Reproducible setup outline: We clone the repository and write down the commit that we tested and the Node/npm versions as well. We look into package.json, the imports in the source files and any existing tests. We install the dependencies (npm ci if lockfile exists, otherwise npm install). We add Jest and write a small smoke test for one of the selected functions and then we run it locally. We set up collectCoverageFrom so the coverage can be measured based on the 10 selected files only and leave **/.internal/** out. We make a LCOV report with Jest for Coveralls. In Part 2, we add a workflow to GitHub Actions that installs the dependencies, it runs the tests and uploads the LCOV report. We won't change library's source code and we will write all the commands and settings to reposetory's README file. The Appendix E shows only a small settings piece that we have not try yet.

3.2 Execution and result reporting

For each test run record include the date/time, branch name, commit hash, Node.js version used, test command, number of tests run, passed, failed, skipped, blocked, planned, executed, expected vs. actual outputs, coverage on selected files, and links to related issues for each test run. A test failure must be initially investigated in terms of the test oracle and the environment before calling it a library bug. During the test phase, keep track of the progress in a concise manner; after the test phase, release a report on test results per functionality, open issues, and scenarios.

3.3 Defect tracking

Use **GitHub Issues** through labels based on severity, priority, functionality, and state. Process: **New → Confirmed → Investigating → Resolved → Retest (if any) → Closed**; an issue other than a defect may be **Rejected** along with the reason for that. "Resolved" means that the defect was resolved, although it does not imply changing the library's code. Severity: severity/impact on the business side (Critical – a matter of safety in totals or lack of function in core functionality; High – significant incorrectness of the result; Medium –

limited error case; Low – minor problem with unusual input handling). Priority: urgency (P1 – urgent problem; P2 – soon; P3 – standard; P4 – low).

Issue Template: Title & Issue number; File/export that got impacted; Test case number; Commit/Environment; Exact input and precondition; Steps involved; Expected result (oracle); Actual result; Logs/Screenshots (if any); Severity/Priority; Assignee & Status; Impact of the issue on scenario; Retest details.

3.4 Metrics, entry/exit criteria and quality gates

Entry: ten files fixed; expected contracts reviewed; runnable test environment and configuration setup; cases with clear inputs and oracles. **Case pass:** actual output is equal to expected contract, including any thrown exceptions. **Blocked:** an unsatisfied pre-requisite stops the case execution; log separately. **Exit:** all intended cases run or justified to be blocked; failures analyzed and logged; coverage collected for selected files only; uncovered branches and open risks analyzed; report generated. Cleanliness of a stage is not reached when there are still important failures present, despite high coverage.

Metric	Calculation / target
Execution rate	Executed / planned × 100; target 100% or documented exceptions
Pass rate	Passed / executed × 100; report observed rate, do not invent a target outcome
Confirmed defects	Count by function, severity and scenario consequence
Statement coverage	Covered statements / measurable statements; **aim 80%**
Branch coverage	Covered branches / measurable branches; **aim 80%** where available

The 80% here refers to **group coverage** and not individual assignment coverage. Create valid, invalid, equivalence class, boundary, and negative test cases; examine unexecuted branches; create test cases or justify the non-executability of some test paths. Coverage is an indication of execution and not correctness.

4. Tests and AI in testing

4.1 Unit-test design principles

Each selected export function is tested individually using deterministic input data and an expected outcome. Equivalence partitioning (of valid/invalid types), boundary value analysis (with empty/singleton/boundary page size), decision branch coverage, negative and regression test cases must be used. Tests should be independent, reproducible and behave-named. The validated contract rather than an e-commerce specific feature will serve as the oracle. Implementation detail testing and test duplication in `.internal` package should not be included. Stubs are to be used only when there is need for external and non-deterministic inputs; otherwise they are not required for pure collection helper functions. In case semantics of actual source code differs from the proposed oracle, check the contract and adjust the test case.

4.2 Detailed test designs for two selected functions

Function A: `chunk.js` — pagination support (S1)

Assumed interface for test design: `chunk(array, size)` takes an array and chunks it into smaller arrays that are continuous and consist of up to `size` elements; the last chunk can be shorter. The actual behavior in case of a default or incorrect size value needs to be confirmed in the repo. **No stubs** are required.

ID	Input	Expected output	Purpose
CH-01	`[1,2,3,4,5]`, size `2`	`[[1,2],[3,4],[5]]`	Non-divisible page count
CH-02	`[1,2,3,4]`, size `2`	`[[1,2],[3,4]]`	Exact division
CH-03	`[]`, size `3`	`[]`	Empty catalogue
CH-04	`[7]`, size `3`	`[[7]]`	Fewer items than page size
CH-05	`[1,2,3]`, size `1`	`[[1],[2],[3]]`	Smallest positive size

ID	Input	Expected output	Purpose
CH-06	`[1,2,3]`, size `5`	`[[1,2,3]]`	Oversized page
CH-07	`['a','b','c']`, size `2`	`[['a','b'],['c']]`	Preserve values/order
CH-08	`[0,null,false]`, size `2`	`[[0,null],[false]]`	Preserve falsy values
CH-09	`[1,2,3]`, size `0`	**Confirm source contract before asserting**	Invalid/boundary size
CH-10	`[1,2,3]`, size `-1`	**Confirm source contract before asserting**	Negative size handling

CH-09/10 should be described first according to the observed contract (without a result, coercion, error, etc.), and then the expected one should be chosen; you should not invent a validation rule for the helper method in your application. **Scenarios linked:** CH-01/02/04/05/06 are about first and last page rendering; S1 additionally requires tests for saving filters between pages.

Function B: `toNumber.js` — numeric conversion (S1, S2, S3)

Assumed interface for test design: `toNumber(value)` performs conversions according to the specified number coercion semantics in the library; however, it **does not** check whether the value is a correct monetary amount or has two decimal places in currency format. Checking for `NaN` should use `Number.isNaN`, not equality. **No stubs** will be required. Test actual source implementation with regard to white space handling, non-decimal formats, and objects.

ID	Input	Expected result	Purpose
TN-01	`'12.50'`	`12.5`	Decimal price string
TN-02	`0`	`0`	Zero remains zero
TN-03	`'-3'`	`-3`	Negative numeric conversion (not acceptance)
TN-04	`'42'`	`42`	Surrounding whitespace
TN-05	`'abc'`	`NaN`	Malformed numeric text
TN-06	`''`	`0`	Empty-string conversion; app must separately reject blank price
TN-07	`null`	`0`	Null conversion; app must separately reject missing price
TN-08	`true`	`1`	Boolean coercion; not valid producer input
TN-09	`'0x10'`	`16`	Nondecimal numeric notation
TN-10	`'1.234'`	`1.234`	More than two decimal places remain unrounded
TN-11	`'Infinity'`	`Infinity`	Non-finite values need separate validation
TN-12	`['7']`	**Confirm source contract before asserting**	Array coercion / unexpected input type

Scenario links: TN-01/04 allow for filtering of prices and parsing of product entries; TN-05/06/07/08/10/11 demonstrate the necessity of validating frontend and backend after number conversion. In S2, calculate manually the cart total (e.g., $2 \times €3.25 + 1 \times €1.40 = €7.90$), while checking formatting/rounding in application level, **not** as `toNumber` promise.

Coverage of the other eight selected files

For `filter`, verify match/mismatch predicate, empty collections and more than one item. For `isEmpty`, compare empty and nonempty arrays, strings and objects. For `add`, verify positive, negative and zero operands; floating point calculations should not use currency-specific rounding conventions. For `countBy`, verify repeating categories, unique entries and empty arrays. For `reduce`, verify presence of an accumulator, empty array, and reducer precedence. For `every`, verify matches of all, one, none, and the empty array special case. For `defaultTo`, verify `null` and `undefined` against `0` and `false` (source). For `slice`, verify negative indices, index out of range, empty result and ordering. Link test cases to corresponding source code and S1–S3 in the Part 2 report.

4.3 Additional testing for the complete e-commerce system

Functional E2E (top business priority): run S1 with filters, no results, optional missing fields and pagination; S2 with quantity changes, empty cart, totals, checkout failure and duplicate submission protection; S3 with valid and invalid records, partial import, duplicates and catalogue verification. Use realistic fixtures and backend persistence verification, not only frontend notifications.

API/integration: validate REST API request/response schema and errors, cart/persistences, producer API permission access, checkout requests, retry strategy on payment provider and preventing duplicate payments. Mock payment provider externally for deterministic failures and run a small sandbox contract test if possible. Find out if bulk import API call is atomic or not before validation.

Security/data integrity: check input validation/XSS, authorization of producer API calls, CSRF if session based, protection from injections, rate limiting and protection of payment data; do not use real customer/card details. Check required fields and two decimal places monetary policy on the server side as the frontend validation can be easily bypassed.

Nonfunctional and usability testing: testing performance with regard to big catalogues, filtering, and batch uploads; test pagination consistency; test search/cart/errors accessibility; test compatibility with browsers/mobility; test fault tolerance for network problems and failure recovery. The tests related to price accuracy, authentication, check out functionality, and data accuracy have top priority over cosmetic tests. They are **recommendations** as the program is not accessible [1].

4.4 AI use, opportunities and safeguards

Actual Phase 1 use: ChatGPT was considered in helping to draft and edit this report, mapping scenario to function, improving the explanation for the prioritization, and listing additional testing risks. The responsibility for verification of all the contracts within the library, tools used for testing, references, diagrams, and the content of the report according to the assignment requirements still lies with the team. Any test cases drafted with the usage of the AI are not considered executed, no test coverage or defect results are fabricated.

Possible Phase 2 application: AI (Claude) can provide equivalence classes, edge cases, and the skeleton of tests using Jest for the ten functions; help to analyze failing tests and uncover branch coverage; and create well-written issue tickets. This would be particularly useful when there is no documentation available for the library being used and also for the edge cases of conversion/empty inputs. Threats that need to be considered in this case would be creating imaginary function contracts, incorrect assertion of expected values, shallow tests which only replicate the implementation, skipping security tests, and leaking proprietary data/code. The mitigations would be reviewing the source code before accepting the suggestions, manually validating the numerical oracles, keeping reviewable tests on GitHub PRs, synthesizing the data, and making the assistance of AI transparent.

5. References

References

1. Tampere University, *COMP.SE.200 Software Testing: Practical 1 — Part 1 Test Plan and submission instructions*, 2026. https://moodle.tuni.fi/mod/assign/view.php?id=4296721&action=view
2. Utility library repository: https://github.com/otula/COMP.SE.200-2024-2025-1
3. Jest documentation: https://jestjs.io/docs/getting-started
4. GitHub Actions documentation: https://docs.github.com/en/actions
5. Coveralls documentation: https://docs.coveralls.io/
6. Node.js documentation: https://nodejs.org/en/learn

Appendix

Appendix A — S1 activity diagram

Appendix B — S2 activity diagram

Appendix C — S3 activity diagram

Appendix D — Candidate prioritisation register

File	Scenarios	Coverage	Impact	Likelihood	Total	Decision
`toNumber.js`	1,2,3	3	3	2	8	Selected
`filter.js`	1	1	3	3	7	Selected
`chunk.js`	1	1	3	3	7	Selected
`isEmpty.js`	1,2,3	3	2	2	7	Selected
`add.js`	2	1	3	2	6	Selected
`countBy.js`	3	1	2	3	6	Selected
`reduce.js`	2	1	3	1	5	Selected
`every.js`	1,3	2	2	1	5	Selected
`defaultTo.js`	1,3	2	2	1	5	Selected
`slice.js`	1	1	2	2	5	Selected
`clamp.js`	2	1	1	3	5	Not selected
`toInteger.js`	2	1	2	1	4	Not selected
`toString.js`	1	1	1	2	4	Not selected
`upperFirst.js`	3	1	1	1	3	Not selected

This documented judgement preserves the group's ten-file decision. Scores are planning estimates, not source-verified measurements.

Appendix E — Planned Jest configuration sketch

```
// Illustrative only: confirm module format and source paths before Part 2.
module.exports = {
  testEnvironment: 'node',
  collectCoverage: true,
  coverageReporters: ['text', 'lcov'],
  collectCoverageFrom: [
    'src/{toNumber,filter,chunk,isEmpty,add,countBy,reduce,every,defaultTo,slice}.js',
    '!src/.internal/**'
  ]
};
```

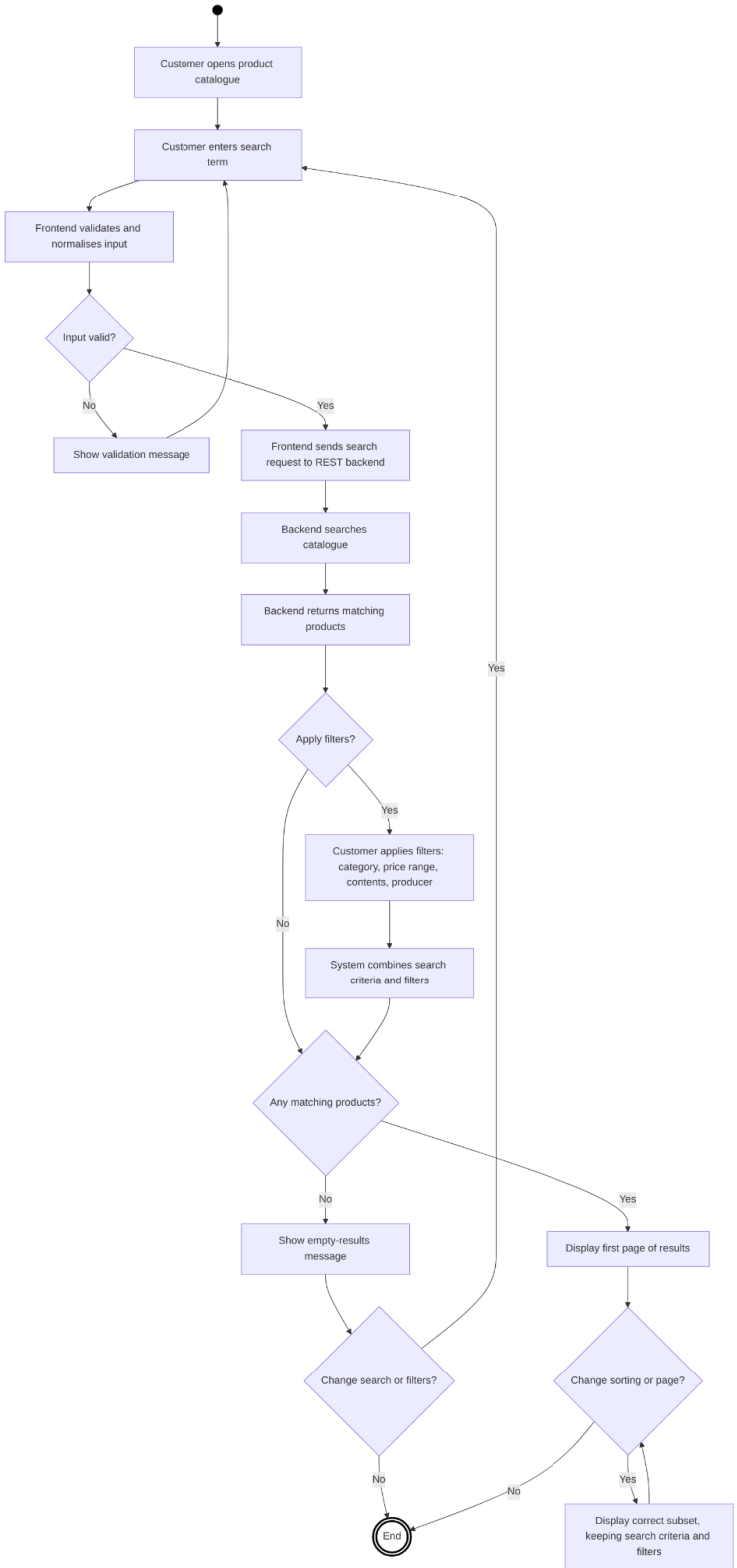


Figure 1: S1 product search activity diagram

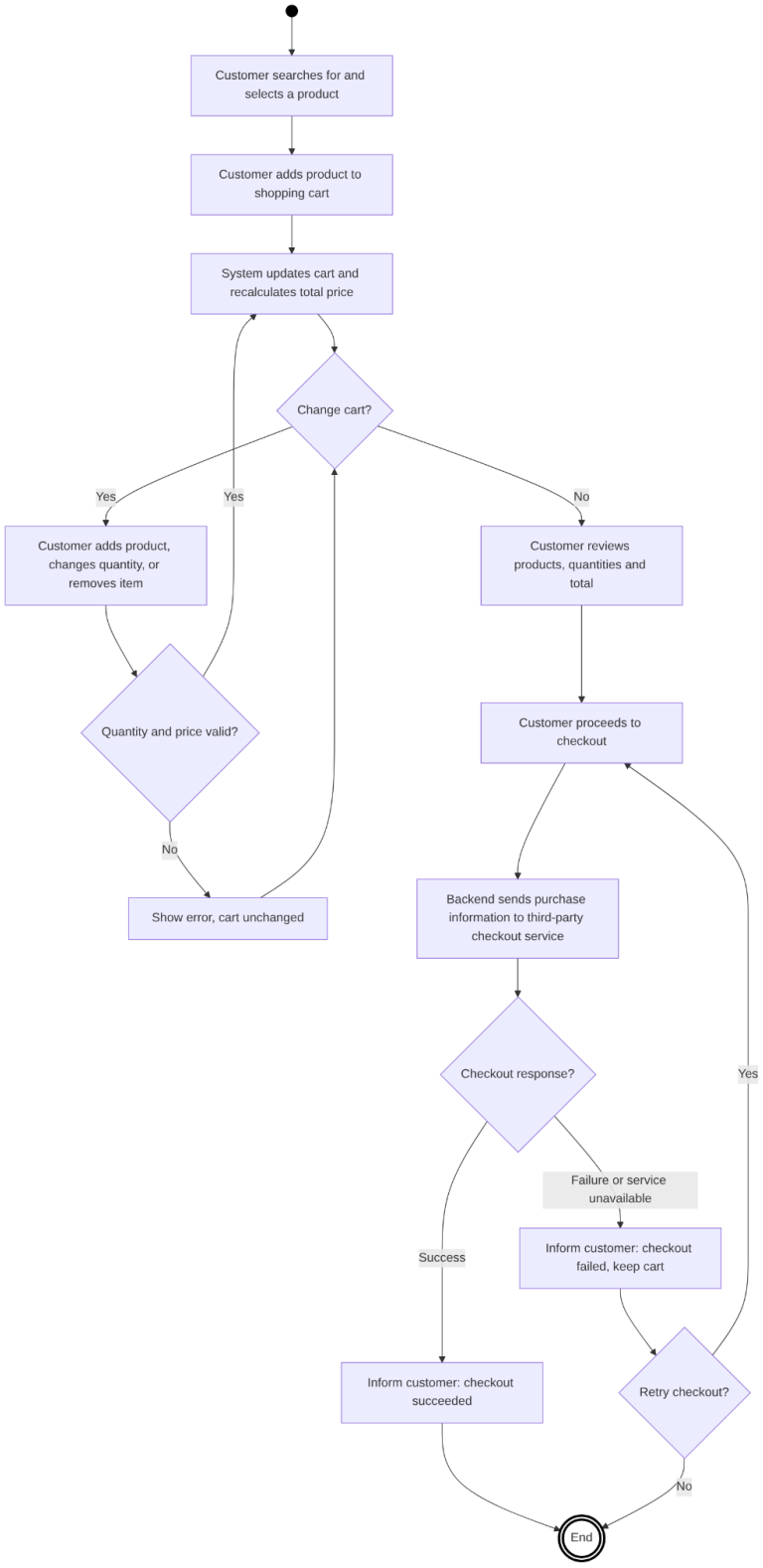


Figure 2: S2 cart and checkout activity diagram

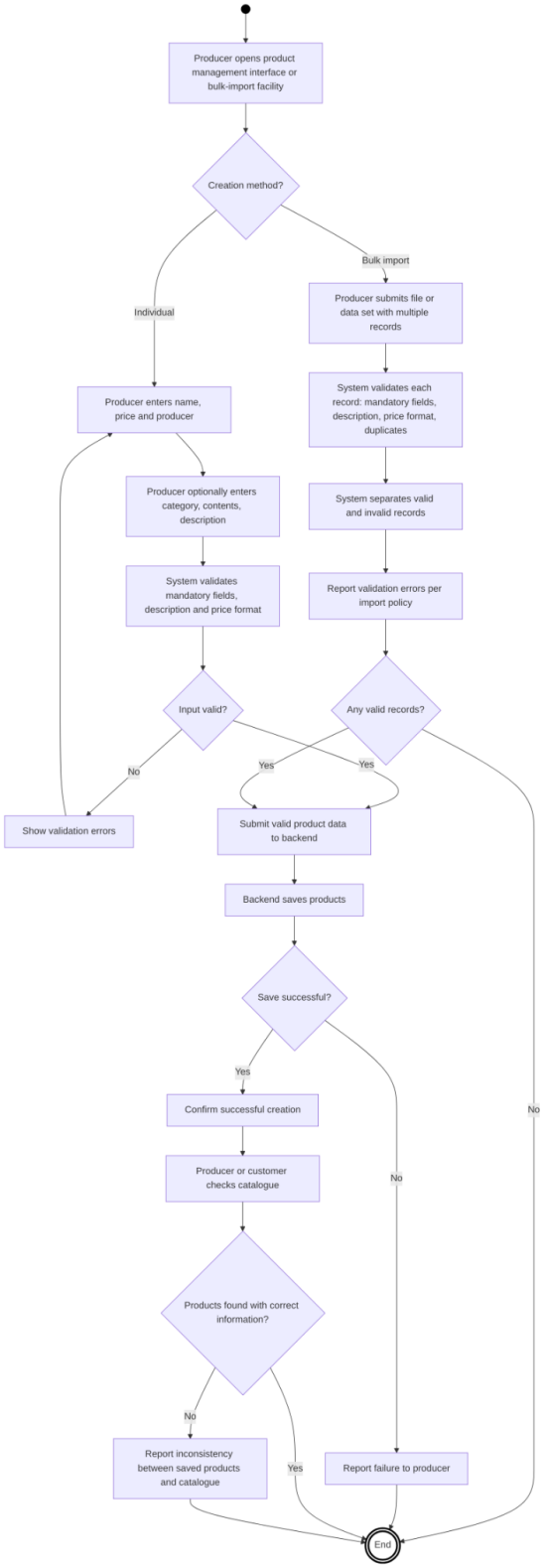


Figure 3: S3 producer import activity diagram